

Doubly Linked List – Length Of Linked List –Space Complexity and Algorithm.

Program :

```
int length()
{
    if (head == nullptr)
    {
        cout << "List is empty." << endl;
        return 0;
    }
    int count = 0;
    Node *temp = head;
    while (temp != nullptr)
    {
        count++;
        temp = temp->next;
    }
    return count;
}
...
case 9:
cout << "Length of the list is: " << length() << endl;
break;
```

Space Complexity

Auxiliary Space

Auxiliary space is the temporary or extra memory used by a function, not including the input data.

In the length function, the new variables created are:

int count: A single integer variable to store the count. An integer occupies a constant amount of space.

Node *temp: A single temporary pointer to traverse the list. A pointer also occupies a constant amount of space.

Since the amount of extra memory is fixed and does not grow with the list's size, the auxiliary space complexity is $O(1)$.

Auxiliary Space: $O(1)$

Input Space

Input space is the memory used to store the function's inputs. We can analyze this from two perspectives.

1. Parameter – only Definition

This view considers only the space taken by the function's direct parameters. The function signature is `int length()`.

The function takes no parameters.

Under this definition, the input space is constant.

Input Space (Parameter-only): $O(1)$

2. Full – Data Definition

This broader view includes the space taken by the entire data structure the function operates on—the doubly linked list.

If the linked list contains n nodes, the total memory it occupies is proportional to n .

Under this definition, the input space is linear with respect to the size of the list.

Input Space (Full-Data): $O(n)$

Total Space Complexity

The total space complexity is the sum of the auxiliary space and the input space. The result depends on which definition of input space you use.

- **Common Interpretation (using parameter-only input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(n)}_{\substack{\downarrow \\ \text{Input}}} = O(n).$$

In most academic and interview contexts, the "space complexity" of a function refers to its auxiliary space. Therefore, the most common and expected answer for this function is $O(1)$. The while loop that traverses the list affects the function's time complexity ($O(n)$), but not its space complexity.

Algorithm: Find Length of List

DLENGTH(START)

1. [CHECK IF LIST IS EMPTY]

If START = NULL, then:

Write: "List is empty."

Return 0 and EXIT.

[End of If structure.]

2. [INITIALIZE COUNTER AND POINTER]

a. Set COUNT := 0 [Initialize a counter to zero].

b. Set PTR := START [PTR is a pointer used to traverse the list].

3. **[TRAVERSE THE LIST AND INCREMENT COUNTER]**

Repeat while $PTR \neq NULL$:

a. Set $COUNT := COUNT + 1$ [Increment the counter for each node].

b. Set $PTR := NEXTLINK[PTR]$ [Move to the next node].

[END of Loop]

4. **[RETURN FINAL COUNT]**

Return $COUNT$.

5. **Exit**
